Lab 2: @RestController

Develop JSON API endpoints for Todo list app.

The website still uses the Thymeleaf HTML pages from Lab 1.

What you will learn

- API development with URL prefix /api/v1/todos.
- How to use the @RestController annotation to develop RESTful API.

Notes

TodoService.java is a Spring bean class, it is containing the ArrayList instance to store a list of todos.

The browser at http://localhost:8080/todos and the API at http://localhost:8080/api/v1/todos will read and write the same ArrayList instance.

For example, when you create a todo using a RESTful API and then refresh the Thymeleaf page, you should see the same new todo.

✓ Step 1. Open the project in Lab 1

Use the folder you built in Lab 1 (com.example.todolist).

- Keep: Todo.java, TodoRequest.java, TodoService.java, TodoController.java, HTML templates.
- Add new:
  - controller/api/v1/TodoRestController.java (working code)
  - controller/api/v2/TodoRestController.java (demo versioning)
- No new Maven dependency: spring-boot-starter-web already includes JSON (Jackson) library.

Why we need API versioning?

A REST API is a contract between backend and clients (mobile, web, third-party). Once published, many clients depend on it. Requirements change, but you cannot break existing clients. Instead of modifying v1 in place, we can maintain multiple versions in parallel.

Spring does not require a special folder. @RestController works in the same package as TodoController. For a tiny app that is acceptable.

Best practice as soon as you have both HTML and JSON: split packages. HTML controllers and API controllers have different return types, errors, and (later) security. A separate package makes that obvious.

Lab 1 @Controller returns a view name, for example "todos/list". Thymeleaf then builds HTML.

Lab 2 @RestController returns a Java object. Spring writes JSON to the HTTP response. There is no Thymeleaf views.

When we need to create v2/v3/…/vN?

Technically required for any breaking change:

- Renaming a field (e.g.: title -> name)
- Removing a field
- Changing a data type
- Changing HTTP status codes
- Changing response structure

Practical consideration: having many clients doesn't cause the need for a new version, it only decides whether versions must run in parallel or all clients can be force-migrated at once.

How to deprecate v1?

- Mark it deprecated via HTTP headers:
  - Deprecation: true
  - Sunset: <date>
- Document it in API docs (Swagger/OpenAPI)
- Notify clients with a clear timeline

✓ Step 2. URL prefix /api/v1/todos

Public APIs usually do not mix with HTML paths like http://localhost:8080/todos. A common pattern is:

```
/api      -> is a REST endpoint returning JSON, not a Thymeleaf HTML page.
/v1       -> version. Later you can add /api/v2 without breaking old clients
/todos    -> resource name, plural noun
/{id}     -> one item

Examples:
  GET    /api/v1/todos    -> get list of todos
  GET    /api/v1/todos/1  -> get a specific todo
  POST   /api/v1/todos    -> create new todo
  PUT    /api/v1/todos/1  -> edit a specific todo
  DELETE /api/v1/todos/1  -> delete a specific todo
```

Why /api? The HTML app stays at /todos. A mobile app or HTTP client calls /api/v1/todos. Security rules (later) can treat /api/** differently from pages.

Why /v1? The first public contract. If you change JSON fields next year, keep /api/v1 and add /api/v2. Old clients still work.

Do not put the version only in a header for this lab. A path version is easy to see and easy to test in the browser.

REST URL rules

- Use a plural noun in the path (/todos), not a verb (/getTodos, /create, /getAll).
- The HTTP method is the verb: GET reads, POST creates, PUT edits, DELETE removes.
- Can use nested paths such as /todos/{id}/comments and query filters such as /todos?completed=true when app requirement changes.

✓ Step 3. TodoRestController (GET)

File: controller/api/v1/TodoRestController.java.

```
package com.example.todolist.controller.api.v1;

import com.example.todolist.dto.TodoRequest;
import com.example.todolist.model.Todo;
import com.example.todolist.service.TodoService;
import jakarta.validation.Valid;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api/v1/todos")
public class TodoRestController {

    private final TodoService todoService;

    public TodoRestController(TodoService todoService) {
        this.todoService = todoService;
    }

    @GetMapping
    public List<Todo> list() {
        return todoService.findAll();
    }

    @GetMapping("/{id}")
    public Todo getOne(@PathVariable Long id) {
        return todoService.findById(id);
    }

    @PostMapping
    public Todo create(@Valid @RequestBody TodoRequest request) {
        return todoService.create(request);
    }

    @PutMapping("/{id}")
    public Todo update(@PathVariable Long id,
                       @Valid @RequestBody TodoRequest request) {
        return todoService.update(id, request);
    }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable Long id) {
        todoService.deleteById(id);
    }
}
```

TodoRestController.java (continued)

```
    @PostMapping
    public Todo create(@Valid @RequestBody TodoRequest request) {
        return todoService.create(request);
    }

    @PutMapping("/{id}")
    public Todo update(@PathVariable Long id,
                       @Valid @RequestBody TodoRequest request) {
        return todoService.update(id, request);
    }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable Long id) {
        todoService.deleteById(id);
    }
}
```

Read the methods

- list / getOne: HTTP GET with no body. Return Todo or List<Todo>. Spring turns them into JSON.
- create: HTTP POST with @RequestBody reads JSON. @Valid runs TodoRequest rules.
- update: HTTP PUT with id is in the URL. Body is the new field values (no id in the DTO).
- delete: HTTP DELETE. Note that HTML forms cannot send DELETE. A REST client can. Return type void -> empty body, status 200.

Example JSON value for Create new todo:

- Field names must match TodoRequest getters/setters.
- dueDate uses ISO date yyyy-MM-dd.

```
{
  "title": "Buy milk",
  "description": "2 liters",
  "dueDate": "2026-09-25",
  "completed": false
}
```

Header when you POST or PUT: Content-Type: application/json

✓ Step 7. Test the API

Start the embedded server and run the Todo list app.

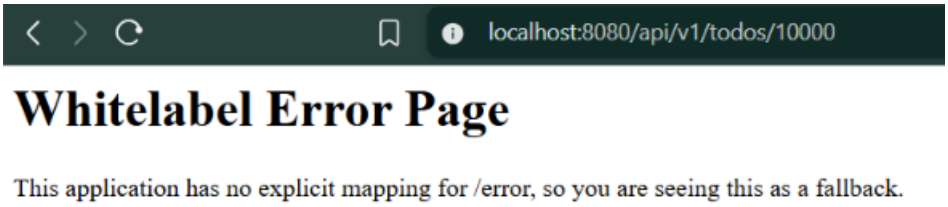
Keep http://localhost:8080/todos open in a browser to double-check between HTML page and RESTful API.

Checking the Todo app with these steps

1. Open http://localhost:8080/api/v1/todos -> JSON array, [] if the list is empty.
2. Create one todo on the HTML page https://localhost:8080/todos, refresh the http://localhost:8080/api/v1/todos you will see the same item appears.
3. POST JSON with a title -> response has id 1 (or the next number).
4. Refresh http://localhost:8080/todos you will see the new row is there because we are using the same ArrayList instance.
5. POST JSON with empty title the server will return error with HTTP status 400.
6. GET http://localhost:8080/api/v1/todos/1 -> one JSON object.
7. PUT http://localhost:8080/api/v1/todos/1 with field “completed” = true -> You will see the GET method and the HTML list both show todo 1 were completed.
8. DELETE http://localhost:8080/api/v1/todos/1 -> that id is gone in JSON and on the HTML page.
9. We can still create and edit as in Lab 1 in the http://localhost:8080/todos page.

✓ Step 8. Handle todo not found exception

When we test the GET method with invalid Todo Id we will get a default error page.
Example endpoint: http://localhost:8080/api/v1/todos/10000



We need to handle to show the Not Found HTTP status 404 error.

There are many ways to do it, one of it is create a custom Exception class.

- Create new exception package: src/main/java/com/example/todolist/exception
- Adding custom exception class: TodoNotFoundException.java

```
package com.example.todolist.exception;

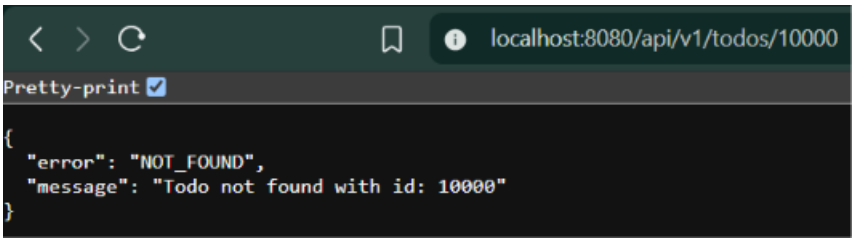
public class TodoNotFoundException extends RuntimeException {
    public TodoNotFoundException(Long id) {
        super("Todo not found with id: " + id);
    }
}
```

- Service throws the new custom Exception

```
public Todo findById(Long id) {
   return store.stream()
               .filter(t -> t.getId().equals(id))
               .findFirst()
               .orElseThrow(() -> new TodoNotFoundException(id));
}
```

- Inside the TodoRestController.java class

```
@ExceptionHandler(TodoNotFoundException.class)
public ResponseEntity<Map<String, Object>> handleNotFound(TodoNotFoundException ex) {
   return ResponseEntity
           .status(HttpStatus.NOT_FOUND)
           .body(Map.of(
                   "error", "NOT_FOUND",
                   "message", ex.getMessage()
               ));
}
```



✓ Step 9. Summary

- The working TodoRestController lives in controller.api.v1 package and calls TodoService only.
- GET /api/v1/todos returns JSON.
- POST / PUT use @Valid @RequestBody TodoRequest.
- A todo created by JSON appears on the Thymeleaf page, and vice versa.
- Lab 1 HTML still works at http://localhost:8080/todos